Appointment Scheduler Technical Progress Report

Software Design 1 SDN150S | Project 12

Prepared for Management | 5 November 2026

https://github.com/ZuluBarbie/AppointmentScheduler

Group 12

Noluthando | 260874574

Zilungile Cwathi | 250513447

Phakamani Nicholas Dlamini | 261059742

Sekete Manthata | 260555770

Liyabona Ndevu | 260173436

Okuhle Ntweni | 260016039

Reneloe Wondeful Phakwago | 260633283

Lisakhanya Songqwaba | 261136437

1 Introduction

Day 4 adds reminders, daily and weekly recurrence, and time conflict detection to the Java appointment scheduler. The application retains appointment editing, deletion, local storage and day, week and month views for Project 12 (Cape Peninsula University of Technology, 2026).

2 Sub questions

How can the scheduler display recurring dates without copying stored records? How can it detect a clash on a future date? How can an in-app reminder avoid appearing twice for the same occurrence?

3 Discussion

Appointment validates the start, end, repeat range and reminder lead time. AppointmentBook stores each series once. It creates Occurrence records for the selected view. Each row refers back to its stored series, so editing or deleting a row affects that series. A repeat end date is inclusive and may be up to 366 days after the start.

Before adding or editing, the book compares overlapping time intervals and checks whether the two appointments occur on a common date. End times are exclusive, which allows adjacent appointments. Validation runs before saving. Changes use a temporary file and request atomic replacement where supported (Oracle, n.d.a). Day 4 can read Day 3 files and defaults their new settings to off.

SchedulerFrame uses the paired NetBeans form for the base layout. The repeat and reminder controls are added in Java. A Swing timer checks upcoming occurrences once per second. Reminders appear while the window is open and are tracked for that session. Java and the JDK support implementation and testing. Git records each working milestone.

4 Design artefacts

4.1 Pseudocode

READ and validate input, repeat end date and reminder lead time

FOR each other appointment with overlapping times

  IF both occur on any candidate date, reject the change

SAVE the new list and refresh the selected view

FOR each upcoming occurrence inside its reminder window

  IF it has not alerted this session, show and mark the alert

4.2 UML class diagram

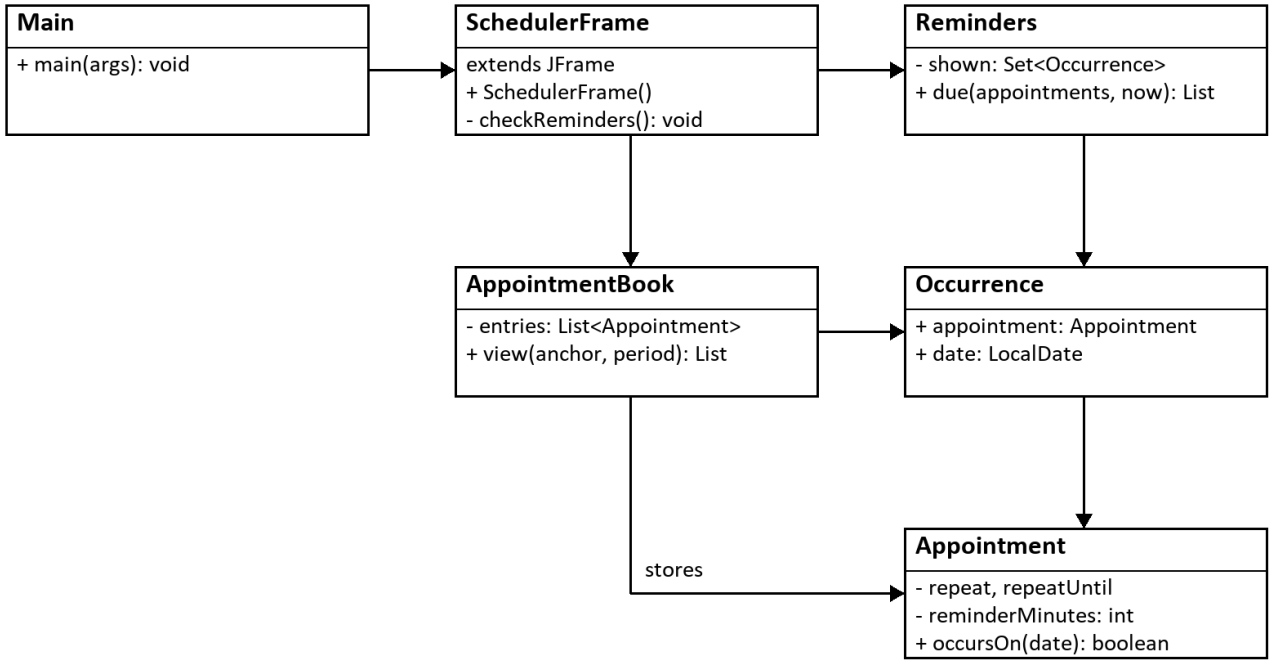


Figure 1. Main opens the Swing frame. AppointmentBook expands stored series into dated rows and Reminders tracks alerts for those occurrences.

5 Results and analysis

The standalone Day 4 JAR built on Windows. All 55 automated checks passed. The 17 new checks cover recurrence boundaries, time clashes, rejected edits, adjacent slots, series deletion, persistence, Day 3 file loading and reminder timing. Existing input, storage and view tests passed.

A Swing test with isolated data displayed a real reminder dialog, selected a repeat row and rejected a duplicate through the Add button. The clash message identified the series and date. The saved file was unchanged. The test captured both rendered windows for inspection.

Reminders depend on an open application. Reopening within the reminder window can display the alert again. Files are local and the program is intended for one user. The Windows build does not establish behaviour on other operating systems.

6 Conclusion

Day 4 adds the planned recurrence, reminders and clash checks. Calendar export and the final submission review are scheduled for 6 November. The recording remains outstanding for the 8 November deadline.

References

Cape Peninsula University of Technology (2026) Software Design 1 SDN150S project brief. Course handout.

Oracle (n.d.a) Files. Java SE 21. Available at: https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/nio/file/Files.html (Accessed: 4 November 2026).

Appendix A Java program excerpts

The complete runnable Java program, paired form and test suite are in demos/day4. The entry point is demo.Main. The excerpts below show the recurrence rule and the conflict check used before saving.

Appointment recurrence

```
public boolean occursOn(LocalDate day) {
    if (day.isBefore(date) || day.isAfter(repeatUntil)) return false;
    return repeat != Repeat.WEEKLY || day.getDayOfWeek() == date.getDayOfWeek();
}
```

AppointmentBook conflict check

```
private void checkConflicts(Appointment candidate, int excludedIndex) {
    for (int i = 0; i < entries.size(); i++) {
        Appointment existing = entries.get(i);
        if (i == excludedIndex || !candidate.getStart().isBefore(existing.getEnd())
                || !existing.getStart().isBefore(candidate.getEnd())) continue;
        Optional<LocalDate> clash = dates(candidate.getDate(), candidate.getRepeatUntil())
                .filter(candidate::occursOn).filter(existing::occursOn).findFirst();
        if (clash.isPresent()) {
            throw new IllegalArgumentException("Overlaps " + existing.getTitle() + " on " + clash.get() + ".");
        }
    }
}
```